

Progettazione Architetturale del SW

Trentin Tommaso

December 19, 2025

Contents

1	Progettazione del SW	2
1.1	Finalità	2
1.2	Decisioni di progettazione	2
1.2.1	Top-Down	2
1.2.2	Bottom-Up	2
2	Progettazione Architetturale	2
2.1	Architettura, requisiti e prestazioni	2
2.1.1	Requisiti non funzionali e stili architetturali	2
2.2	Progettazione architetturale e agilità	3
2.3	Vantaggi progettazione esplicità	3
3	Pattern Architetturali	3
3.1	Concetto di pattern	3
3.2	Pattern multi-layer	3
3.3	Pattern repository	3
3.3.1	Vantaggi	4
3.3.2	Svantaggi	4
3.4	Pattern client-server	4
3.4.1	Vantaggi	4
3.4.2	Svantaggi	5
3.4.3	Componenti	5
3.4.4	Esempi	5
3.5	Pattern pipe-and-filter	5
3.5.1	Vantaggi	5
3.5.2	Svantaggi	5
3.6	Pattern model-view-controller (MVC)	6
3.6.1	Aggiornamento dati	6
3.6.2	Selezione schermata	7

1 Progettazione del SW

La progettazione architeturale si colloca nella fase di Progettazione del sistema ed ha un **ruolo chiave nello sviluppo**, rappresentando il collegamento critico tra ingegneria dei requisiti e progettazione di dettaglio dei componenti.

1.1 Finalità

Progettare = seguire un processo di risoluzione mirando a descrivere un modo:

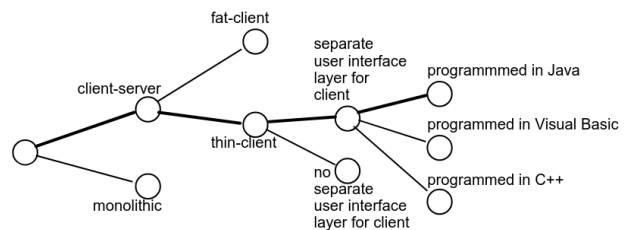
- per implementare requisiti funzionali;
- rispettando vincoli dei requisiti non funzionali;
- in conformità con principi di buona qualità.

1.2 Decisioni di progettazione

Progettista affronta una serie di problemi di progettazione, ognuno di questi ha normalmente alcune soluzioni alternative (**design options**).

Progettista effettua una serie di **design decision** per risolvere il problema scegliendo la *migliore* alternativa possibile in base a requisiti sistema e background / esperienza architetti.

Design space: insieme possibili progetti che saranno implementabili scegliendo tra varie alternative.



1.2.1 Top-Down

- Si parte da progettazione della struttura ad altissimo livello del sistema, per poi procedere gradualmente più nel dettaglio riguardo aspetti di più basso livello (es. definizione architettura SW e tipo di DB impiegato -> definire formato di dati, algoritmi usati, interfacce utente);
- Necessaria per definire una buona struttura per il sistema.

1.2.2 Bottom-Up

- Si parte progettando componenti di basso livello e si decide poi come collegarli per ottenere i componenti di livello più alto;
- Serve a progettare componenti riusabili in altri punti del sistema.

2 Progettazione Architeturale

Si occupa di comprendere come organizzare il sistema SW e progettarne la sua struttura complessiva, identifica i componenti strutturali di un sistema e le loro relazioni / interfacce; ha come output un modello che descrive l'organizzazione del sistema e la comunicazione tra i suoi componenti.

2.1 Architettura, requisiti e prestazioni

Architettura SW è importante in quanto influisce (o addirittura pone vincoli) su prestazioni / robustezza / distribuzione / manutenibilità di un sistema.

2.1.1 Requisiti non funzionali e stili architettureali

- **Performance:** preferenza per componenti grandi, localizzando operazioni critiche e minimizzando comunicazioni;
- **Protezione:** utilizzo di architettura a strati, posizionando risorse critiche negli strati interni e ponendo convalide ad ogni strato;
- **Sicurezza funzionale:** concentrazione funzionalità critiche per sicurezza in numero limitato di sottosistemi per fornire meccanismi di protezione riducendo i costi;

- **Disponibilità:** utilizzo di componenti ridondanti e meccanismi per tolleranza ai guasti;
- **Manutenibilità:** utilizzo componenti piccoli facilmente sostituibili / modificabili.

2.2 Progettazione architetturale e agilità

Attività di progettazione architetturale può scontrarsi con principi agili di **documentazione minima** e **refactoring** (possibile rifattorizzare singoli componenti ma per l'intera architettura in quanto ciascun cambiamento può interessare diversi componenti).

2.3 Vantaggi progettazione esplicità

- Fornisce presentazione del sistema ad alto livello che consente confronto tra stakeholders;
- Durante progettazione viene analizzata conformità del sistema ai requisiti non funzionali;
- Modello può essere riusato per sistemi con requisiti simili.

3 Pattern Architetturali

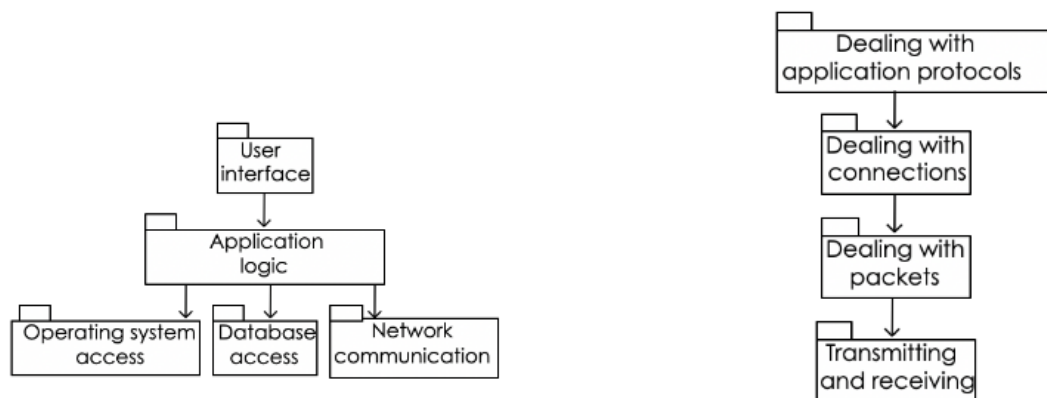
3.1 Concetto di pattern

Soluzioni a problemi di progettazione comuni e ricorrenti che si incontrano in applicazioni reali, un pattern architetturale descrive l'organizzazione del sistema che è stata utilizzata in diversi sistemi SW. Vantaggiosi in quanto permettono:

- **Riuso** di conoscenza ed esperienza di progettazione in quanto problemi sono raramente nuovi e unici;
- **Presentazione e comunicazione** in quanto stabiliscono una terminologia comune e condivisa, forniscono una prospettiva di alto livello del sistema e permettono comprensione / discussione del progetto senza dover gestire troppo presto i dettagli.

3.2 Pattern multi-layer

Decomposizione *gerarchica* del sistema in un insieme ordinato di layer (sottoinsiemi che forniscono servizi correlati), ogni layer può essere a sua volta stratificato in layer più astratti. Fornisce:



- **separazione:** solitamente elementi che forniscono servizi correlati sono raggruppati nello stesso layer, separati da altri servizi;
- **indipendenza:** possibile progettare uno strato indipendentemente da altri (rispettando le interfacce).

Grazie a separazione e indipendenza, se si modifica interfaccia / si aggiungono funzionalità ad uno strato, solo strati adiacenti vengono influenzati. Questa è quindi una soluzione ideale per sistemi multipiattaforma (dipendenze della macchina localizzate in strati specifici).

3.3 Pattern repository

Utilizzato in casi di diversi sottosistemi con grandi quantità di dati da condividere e memorizzare per molto tempo, questi vengono salvati / gestiti in un database centrale (**repository**) il cui accesso è consentito a tutti i sottosistemi.

- Tutti i sottosistemi devono operare con un modello dati concordato;

- Repo è accessibile a tutti i componenti del sistema;
- Componenti scambiano dati solo mediante la repo.



3.3.1 Vantaggi

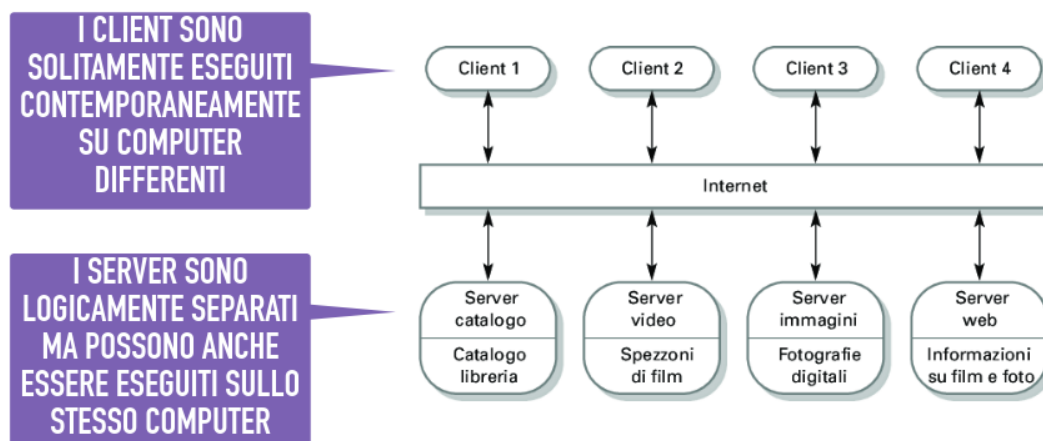
- Possibile indipendenza tra i componenti (non devono trasmettere dati direttamente tra loro e non devono per forza sapere dell'esistenza degli altri);
- Repo gestisce tutti i dati concorrentemente (controllo accessi / backup);
- Semplice integrazione di nuovi componenti che si conformano al modello dati condiviso

3.3.2 Svantaggi

- Rende difficile la distribuzione su più macchine (ridondanza e inconsistenza a causa del dover mantenere coerenti / aggiornate copie degli stessi dati);
- Repository è **single point of failure** (tutti suoi problemi influiscono su tutto il sistema);
- Difficile integrazione di componenti non conformi al modello dati comune.

3.4 Pattern client-server

Architettura **distribuita** in cui ciascun servizio viene fornito da un server separato al cui i client accedono per utilizzarne i servizi.



3.4.1 Vantaggi

- Server possono essere distribuiti su una rete;
- Server e servizi sono indipendenti / possono essere modificati senza intaccare altre parti del sistema;
- Facile aggiunta / integrazione di nuovi server al sistema.

3.4.2 Svantaggi

- Ciascun servizio costituisce un **single point of failure**, ma potrebbe essere replicato su più server;
- Prestazioni potrebbero dipendere da canali di comunicazione (congestione rete);
- Possibili problemi di gestione in caso di server proprietari di organizzazioni diverse.

3.4.3 Componenti

Ogni applicazione viene suddivisa logicamente in 3 parti:

- **Presentazione** - gestione UI (eventi grafici, controlli su input...);
- **Logica applicativa** - logica e regole che definiscono elaborazione dei dati per rispondere alle richieste utente;
- **Gestione dei dati persistenti** - memorizzazione / gestione dei dati a lungo termine.

La distribuzione fisico-logica di queste componenti porta a classificazione delle architetture:

- **2-Tier**: suddivisa in *client* e *server*, la logica applicativa può essere distribuita tra i due;
- **3-Tier**: i tre livelli sono separati - la presentazione su *client*, logica applicativa su un *application server* e gestione dei dati su un *database server* (scelta tipica per app web scalabili);
- **n-Tier**: utilizza ulteriori divisioni logico-fisiche aggiungendo livelli come *Cache Layer* per ottimizzare prestazioni e *Security Layer* per autenticazioni / autorizzazioni centralizzate (ideale per sistemi complessi - microservizi o app su cloud).

3.4.4 Esempi

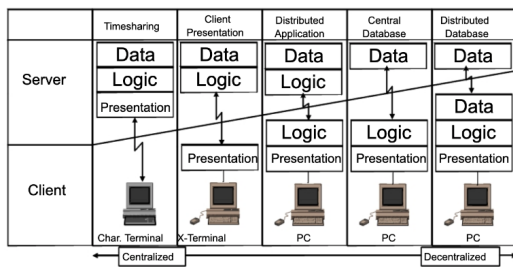


Figure 1: Esempio 2-Tier

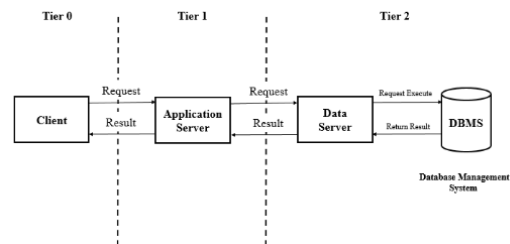
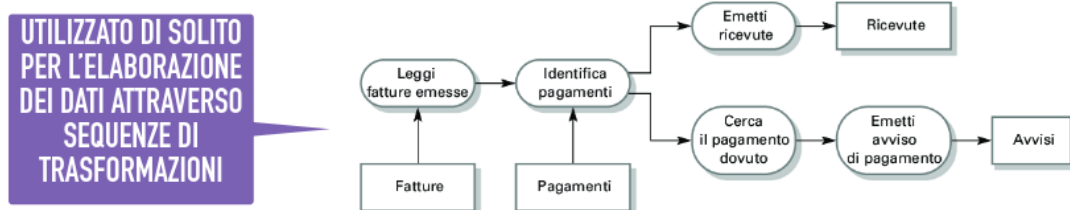


Figure 2: Esempio 3-Tier

3.5 Pattern pipe-and-filter

Ciascun componente di elaborazione (filtro) è discreto e fornisce un output effettuando una trasformazione (in sequenza o in parallelo) dei dati in input. I dati fluiscono (singoli o in batch) attraverso una sequenza di componenti per essere elaborati.



3.5.1 Vantaggi

- Evoluzione del sistema semplice mediante modifica / aggiunta di trasformazioni;
- Possibile riutilizzo trasformazioni;
- Possibile progettare separatamente componenti.

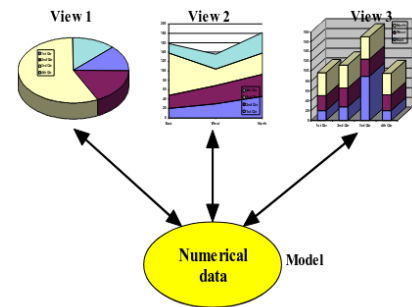
3.5.2 Svantaggi

- Trasformazioni successive devono rispettare formato concordato per trasferimento dati;
- Possibile non poter riusare componenti dati incompatibili / introdurre overhead per traduzione formato;

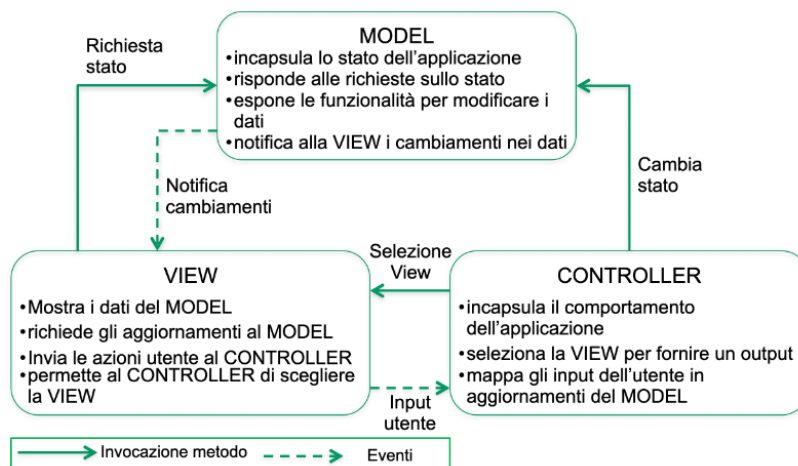
- Interazione con utenti limitata (interfacce con utenti solitamente non forniscono input come semplici flussi sequenziali continui).

3.6 Pattern model-view-controller (MVC)

Nasce da necessità di visualizzare dati generici mediante GUI usando rappresentazioni diverse, consente realizzazione di architettura che permette separazione dei componenti di presentazione da quelli che gestiscono i dati.

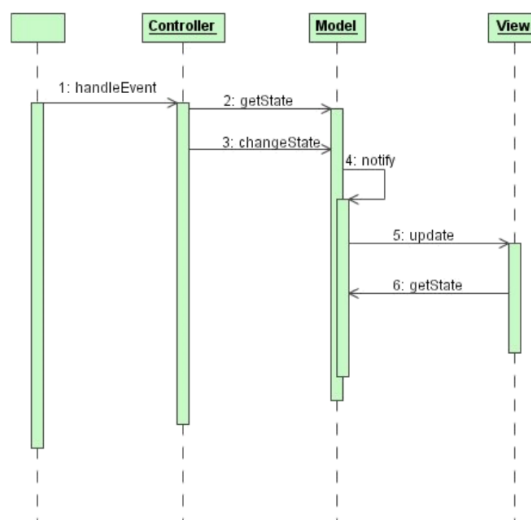


- **model** contiene classi in cui istanze rappresentano dati da visualizzare / manipolare (forniscono anche i metodi per fare ciò);
- **view** contiene oggetti per presentare dati contenuti nel *model* all'utente;
- **controller** contiene oggetti che controllano / gestiscono interazione dell'utente con *view* e *model*.



3.6.1 Aggiornamento dati

1. In risposta a evento *x* il metodo **handleEvent()** viene invocato sul *Controller*;
2. *Controller* esamina stato di *Model* e invoca suo metodo per cambiare il suo stato (**changeState()**);
3. *Model* cambia lo stato e notifica lo notifica a tutte le *View* registrate (**notify()**);
4. Ogni *View* registrata viene notificaa invocando suo metodo **update()**, la *View* esamina lo stato di *Model* mediante metodo **getState()**.



3.6.2 Selezione schermata

1. Utente seleziona una pagina da mostrare mediante *View*;
2. *View* delega a *Controller* tramite invocazione del suo metodo **processSelectPage()**;
3. *Controller* decide e/o compone pagina da visualizzare, comunicandola poi a *View* invocando suo metodo **showSelectPage()**;
4. *View* costruisce e presenta all'utente la schermata richiesta, con eventuale controllo su possibili modifiche in *Model* invocando metodo **getState()**.

